

Building the Outbound Machine: A Signal-Driven ICP Scoring and Outreach Engine for DuPont Tedlar

Mohsin Khawaja

InstaLILY AI Case Study, Go-To-Market Engineer

Aug 27, 2026

Abstract

Outbound only works when effort concentrates on the accounts most likely to convert, and that requires a scoring model a revenue team can operate rather than a black box it has to trust. This case study builds the engine end to end for DuPont Tedlar's Graphics and Signage unit: account sourcing from three channels, enrichment from the companies' own sites, an ICP scoring model expressed as data, contact discovery across a provider stack, and drafted outreach bound to cited evidence, all surfaced in a dashboard where every number traces to the page it came from. The design bias is deliberate. Scoring, deduplication, retries, persistence and provider selection are ordinary Python, while the language model is confined to reading unstructured pages and writing prose over facts the pipeline has already verified. The result is a score that is reproducible, decomposable and arguable. The system sourced 97 companies, enriched 90, qualified 16 as tier A or B, identified 32 decision-makers and produced 29 outreach drafts, at \$0.63 across 108 model calls, or \$0.039 per qualified lead. Missing data lowers confidence instead of being inferred, and 44 handled failures were recorded as data rather than ending the run.

Introduction

DuPont Tedlar is a protective film sold into graphics and signage applications where ultraviolet exposure, weathering and warranty duration determine material choice. The commercial question is which fabricators, converters and print manufacturers are worth a rep's time, and the answer is not in any single database. It sits in a company's own account of its durability claims and application areas, and in whether it exhibits alongside Tedlar's audience at events such as ISA Sign Expo, FESPA and PRINTING United. Assembling that list by hand is slow and inconsistent, which is exactly the shape of problem worth building a machine for.

The tempting build is a swarm of cooperating model-driven agents that discover, evaluate and decide. This work rejects that design for a commercial reason rather than an academic one. A score produced by a model cannot be disputed by the account executive who has to act on it, changes between runs on identical inputs, and leaves nowhere to attach a fix when a lead turns out wrong. A GTM system has to be operable: the same inputs must produce the same prioritization on Monday as they did on Friday, and when a rep says a lead is bad, someone has to be able to open the score and find the component responsible. The contributions are an ICP scoring model expressed as data in one configuration file, a per-component breakdown carrying the source URL behind each component, an enrichment and contact stack behind a single interface, guardrails that reject model output citing evidence the pipeline did not supply, and an error-isolation primitive that degrades per record rather than per stage. Section 2 covers the agent workflow, Section 3 the data processing, and Section 4 the implementation results.

AI Agent Workflow

The pipeline runs seven stages, presented in the interface as six named agents: event and association discovery, company sourcing, deduplication and normalization, enrichment, qualification, contact discovery and outreach drafting. Orchestration is deterministic, and each agent is a stage with declared inputs, a persistence boundary and its own failure isolation.

Qualification assigns a 0 to 100 score across five weighted components: industry fit (30 points), application fit (25), company size (15), event engagement (15) and pain-point alignment (15). Accounts scoring at least 75 are tier A, at least 60 tier B, and at least 40 tier C; below that they fall outside the ICP. The vocabulary, weights and thresholds are data in a single file, so retargeting the model at a different segment or business unit is a configuration edit rather than a change to any stage. The model never produces the number. It receives the finished breakdown and the evidence list and writes the rationale, which is a presentation task over a decided result. Confidence is computed separately and measures how much of the score is URL-backed, penalized for a missing website or unknown size, so a thin record reads as thin instead of quietly scoring like a researched one.

Contact discovery is a provider chain behind one interface: a public-web provider that follows a site's own navigation to leadership pages, a deterministic mock provider whose records always render at zero confidence with an explicit badge, and credential-gated Clay and Sales Navigator providers implemented against their real call shapes, with Apollo supplying firmographics. Adding a provider means implementing the interface and gating it on configuration, with no caller changes. Candidates are ranked by functional fit rather than seniority alone, since seniority ordering places a finance director above the product manager who actually specifies materials. Two AI-native workflows sit on top: an on-demand prospect research report that assembles a pre-call briefing with talking points and likely objections, and outreach drafting that requires the model to return both its hook fact and the URL for that fact, with the draft rejected when the URL is not one the pipeline supplied. Drafts export as .eml or open directly in Gmail, so the handoff to a rep is a click rather than a copy and paste.

Data Processing

Accounts arrive through three channels that fail in different ways: exhibitor directories, ICP-shaped search queries, and a curated roster of category leaders. The failure of any one degrades recall rather than emptying the pipeline. Records are deduplicated on registrable domain first and fuzzy legal name second, which is the association logic that keeps the same company from entering twice under two spellings. Every HTTP response is written to a content-addressed cache keyed by URL hash, with a sidecar recording status and timestamp. That cache is what makes a replay reproducible, and keyed search providers were routed through it after they were found to bypass it and silently re-search live.

Enrichment fetches each company's own site and follows its navigation to product and technical pages, because the durability and warranty language carrying the qualifying signal rarely appears on a homepage. Extracted text is retained at a deliberately generous cap, since the keyword scan driving the score is nearly free while the model sees only the leading portion. The extraction prompt is null-preferring and forbidden from inferring, and a monetary figure is accepted as revenue only when a revenue word appears within sixty characters of it, which stops a phrase such as a multi-billion-dollar market from being recorded as company turnover. All model output is validated against a schema before it can reach the database, with one repair round-trip permitted before the record is degraded. Per-record work is wrapped in an error-isolation primitive that turns expected failures into stored error rows and lets the run continue, so a provider outage costs one record rather than one stage.

Implementation Results

A full run sourced 97 companies and enriched 90 from their own sites, qualifying 16 as tier A or B, six tier A and ten tier B, across 10 discovered events and associations. Contact discovery returned 32 named decision-makers and outreach produced 29 drafts, of which 28 were model-written and one fell back to a labelled template. The run consumed 108 model calls at \$0.63, or \$0.039 per qualified lead, and recorded 44 handled failures. The test suite is 165 hermetic tests that blank every credential and perform no network access. Everything is exposed through a dashboard with score breakdowns, clickable evidence, contact cards, an outreach editor and a dedicated error tab, deployed on Vercel as a read-only snapshot.

The most useful evaluation was adversarial rather than aggregate: forming a hypothesis about the single example account named in the brief, then testing it. That surfaced three defects inspection had missed. The pain-point vocabulary matched too literally and failed on a company that discusses warranty extensively without using the exact phrase sought. Enrichment consumed its firmographics quota in alphabetical order, so accounts late in the corpus were never enriched. A name guard rejected precisely the pages that answered the qualification question. Fixing these moved the example account from 48 to 73 and raised qualified leads across the corpus from 6 to 16. Separately, at a lower text-retention cap the median company was truncated exactly at the boundary and pain-point alignment read zero on 96 of 116 records, which is a reminder that a metric reading zero everywhere is usually a bug rather than a finding.

Two numbers are reported honestly rather than cleanly. Live runs are not reproducible because search ranking drifts, and the example account has scored 48, 61 and 73 across executions, which is why demonstration uses a cached replay. None of the 32 contacts carries a verified email address, since the available people-search tier does not expose them, and the interface says so instead of constructing a plausible address.

Conclusion

The system is a working outbound engine whose prioritization is reproducible, decomposable and cheap, and in which a language model contributes only where the input is genuinely unstructured. Its constraints are stated rather than hidden. Exhibitor directories on a common event platform are JavaScript-gated and refuse server-side clients, recorded as a miss rather than circumvented, since a licensed source is the correct remedy. Without a paid search key, recall on category leaders is thin, which is why the curated roster channel exists. Most private companies do not publish revenue, so the size component is frequently zero.

The next two builds are the ones that turn a scoring engine into a revenue engine. The first is signal detection, moving from whether an account fits to why it is worth contacting this week, which requires a longitudinal record the system has only begun to accumulate. The second is weighting the components against closed-won outcomes instead of by hand, so the model learns from what actually converts and the measurement runs from first touch through to ARR. Both are designed and deliberately unbuilt, because shipping a weighting model before there is anything to weight it against would be exactly the kind of clean number this system is built to avoid.

Supplementary Materials

GitHub Repository: <https://github.com/mohsin-khawaja/instalily-case-study>

Live Dashboard: <https://instalily-case-study-seven.vercel.app/>

Presentation (process and prototype): docs/Tedlar-Lead-Agent.pptx